EXP NO: **IMPLEMENTATION OF SYMBOL TABLE**
DATE:

AIM: To implement a program that generates a Symbol Table by identifying variables and their corresponding data types from a given C source file using Java.

ALGORITHM:

1. Start the program.
2. Create an empty symbol table using a HashMap to store variable names and their data types.
3. Read the input C program line by line using BufferedReader.
4. Ignore empty lines in the source file.
5. Detect the beginning of the main() function.
6. Process only the lines present inside the main() function.
7. Use regular expressions to identify variable declarations such as int, float, double, char, and long.
8. Extract the data type and variable names from the declaration.
9. If multiple variables are declared in a single statement, split them and store each variable separately.
10. Ignore keywords like main.
11. Insert each variable and its corresponding data type into the symbol table.
12. After reading the entire file, display the contents of the symbol table in tabular form.
13. Stop the program.

CODE:
variables.c

```
#include <stdio.h>
int main() {
   int c, a;
   float sum;
   printf("Variables declared.\n");
   return 0;
}
```

SymbolTableGenerator.java

```
import java.io.*;
import java.util.*;
import java.util.regex.*;

public class SymbolTableGenerator {

   public static void generateSymbolTable(String filePath) {
      HashMap<String, String> symbolTable = new HashMap<>();
```

```
        try {
            BufferedReader reader = new BufferedReader(new FileReader(filePath));
            String line;
            boolean insideMain = false;
            String regex = "\\b(int|float|double|char|long)\\s+([a-zA-Z_][a-zA-Z0-
9_]*(\\s*(,\\s*[a-zA-Z_][a-zA-Z0-9_]*))*)\\s*(;|$)";
            Pattern pattern = Pattern.compile(regex);

            while ((line = reader.readLine()) != null) {
                line = line.trim();


                if (line.isEmpty()) {
                    continue;
                }


                if (line.contains("int main(")) {
                    insideMain = true;
                    continue;
                }


                if (insideMain && line.contains("}")) {
                    insideMain = false;
                    continue;
                }


                if (insideMain) {
                    Matcher matcher = pattern.matcher(line);

                    while (matcher.find()) {
                        String type = matcher.group(1);
                        String variables = matcher.group(2);


                        if (!variables.equals("main") && !variables.isEmpty()) {
                            String[] varList = variables.split(",");
                            for (String var : varList) {
                                var = var.trim().replace(";", "");
                                if (!var.equals("main") && !var.isEmpty()) {
                                    symbolTable.put(var, type);
                                }
                            }
                        }
                    }
                }
            }
```

```
        reader.close();

        System.out.println("Variables\tDatatype");
        System.out.println("------------------------");
        for (Map.Entry<String, String> entry : symbolTable.entrySet()) {
            System.out.println(entry.getKey() + "\t\t" + entry.getValue());
        }

    } catch (IOException e) {
        System.err.println("Error reading the C file: " + e.getMessage());
    }
}

public static void main(String[] args) {
    if (args.length != 1) {
        System.out.println("Usage: java SymbolTableGenerator <path-to-c-file>");
        return;
    }

    String filePath = args[0];
    generateSymbolTable(filePath);
}
```

OUTPUT:

```
it23a41@nlpds-lab:~$ vi variables.c
it23a41@nlpds-lab:~$ cc variables.c
it23a41@nlpds-lab:~$ java SymbolTableGenerator vari
ables.c
Variables       Datatype
------------------------
a               int
c               int
d               float
sum             float
it23a41@nlpds-lab:~$
```

RESULT: Hence, the implementation of the Symbol Table is successfully completed.

EXP NO: **IMPLEMENTATION OF WORD ANALYSIS**
DATE:

AIM: To perform **word analysis** on a natural language corpus using **NLTK**, including corpus exploration, stop word removal, vocabulary creation, word frequency analysis, and slang expansion.

ALGORITHM:

1. Import the required **NLTK libraries** and download necessary resources.
2. Load the **Brown corpus** from NLTK.
3. Display **sample words and sentences** from the corpus.
4. Calculate:
    a. Total number of words **including punctuation**
    b. Total number of words **excluding punctuation**
    c. Total number of sentences in the corpus
5. Remove **punctuation** and **stop words** from the corpus.
6. Create a **vocabulary** consisting of unique words from the filtered corpus and sort it alphabetically.
7. Compute the **frequency of each word** in the vocabulary and store it in a dictionary.
8. Display **three sample words** for each starting alphabet from the vocabulary.
9. Define a slang dictionary and **expand casual slangs** into their formal forms within sentences.

QUESTIONS & OUTPUT:

1. **Import a corpus from the NLTK library and perform word analysis using the Brown corpus. Also, explore any one other corpus available in NLTK.**

```
!pip install nltk
```

```
Requirement already satisfied: nltk in /usr/local/lib/python3.12/dist-packages (3.9.1)
Requirement already satisfied: click in /usr/local/lib/python3.12/dist-packages (from nltk) (8.3.1)
Requirement already satisfied: joblib in /usr/local/lib/python3.12/dist-packages (from nltk) (1.5.3)
Requirement already satisfied: regex>=2021.8.3 in /usr/local/lib/python3.12/dist-packages (from nltk) (2025.11.3)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from nltk) (4.67.1)
```

```
import nltk
nltk.download('brown')
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('gutenberg')
```

```
[nltk_data] Downloading package brown to /root/nltk_data...
[nltk_data]   Unzipping corpora/brown.zip.
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt.zip.
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
[nltk_data] Downloading package gutenberg to /root/nltk_data...
[nltk_data]   Unzipping corpora/gutenberg.zip.
True
```

2. **Display ten sample words or sentences from the selected corpus.**

```
from nltk.corpus import brown, gutenberg

# Brown corpus
brown_words = brown.words()
brown_sents = brown.sents()

# Another corpus (Gutenberg)
gutenberg_words = gutenberg.words()
```

```
print("10 sample words from Brown corpus:")
print(brown_words[:10])

print("\n10 sample sentences from Brown corpus:")
for sent in brown_sents[:10]:
    print(" ".join(sent))
```

```
10 sample words from Brown corpus:
['The', 'Fulton', 'County', 'Grand', 'Jury', 'said', 'Friday', 'an', 'investigation', 'of']

10 sample sentences from Brown corpus:
The Fulton County Grand Jury said Friday an investigation of Atlanta's recent primary election pr
The jury further said in term-end presentments that the City Executive Committee , which had over
The September-October term jury had been charged by Fulton Superior Court Judge Durwood Pye to in
`` Only a relative handful of such reports was received '' , the jury said , `` considering the w
The jury said it did find that many of Georgia's registration and election laws `` are outmoded o
It recommended that Fulton legislators act `` to have these laws studied and revised to the end o
The grand jury commented on a number of other topics , among them the Atlanta and Fulton County p
Merger proposed
However , the jury said it believes `` these two offices should be combined to achieve greater ef
The City Purchasing Department , the jury said , `` is lacking in experienced clerical personnel
```

3. **Calculate the size of the corpus by determining:**
   a. **The total number of words including punctuation**
   b. **The total number of words excluding punctuation**
   c. **The total number of sentences in the corpus**

```
total_words_incl_punct = len(brown_words)
print (total_words_incl_punct)
```

```
1161192
```

```
import string

words_no_punct = [w.lower() for w in brown_words if w.isalpha()]
total_words_excl_punct = len(words_no_punct)
print(total_words_excl_punct)
```

```
981716
```

```
total_sentences = len(brown_sents)

print("Words (including punctuation):", total_words_incl_punct)
print("Words (excluding punctuation):", total_words_excl_punct)
print("Number of sentences:", total_sentences)
```

```
Words (including punctuation): 1161192
Words (excluding punctuation): 981716
Number of sentences: 57340
```

4. **Remove stop words from the corpus using an appropriate stop word list.**

```
from nltk.corpus import stopwords

stop_words = set(stopwords.words('english'))

filtered_words = [w for w in words_no_punct if w not in stop_words]

print("Sample words after stopword removal:")
print(filtered_words[:20])
```

```
Sample words after stopword removal:
['fulton', 'county', 'grand', 'jury', 'said', 'friday', 'investigation', 'recent', 'primary', 'election', 'produced',
```

5. **Using the filtered output obtained after stop word removal:**
   a. **Create a vocabulary consisting of unique words from the corpus in sorted order**

b. **Calculate the frequency of each word in the vocabulary and store the result in the form of a dictionary**

```
vocabulary = sorted(set(filtered_words))
print("Vocabulary size:", len(vocabulary))
```

```
Vocabulary size: 40097
```

```
from collections import Counter

word_freq = Counter(filtered_words)

# Convert to normal dictionary if needed
freq_dict = dict(word_freq)

# Sample output
print("Sample word frequencies:")
print(list(freq_dict.items())[:10])
```

```
Sample word frequencies:
[('fulton', 17), ('county', 155), ('grand', 48), ('jury', 67),
```

6. **Display three sample words from the vocabulary for each starting alphabet.**

```python
from collections import defaultdict

letter_samples = defaultdict(list)

for word in vocabulary:
    first_letter = word[0]
    if len(letter_samples[first_letter]) < 3:
        letter_samples[first_letter].append(word)

# Display
for letter in sorted(letter_samples.keys()):
    print(f"{letter.upper()} : {letter_samples[letter]}")
```

```
A : ['aa', 'aaa', 'aaawww']
B : ['b', 'babatunde', 'babbiting']
C : ['c', 'cab', 'cabana']
D : ['da', 'dabbed', 'dabbing']
E : ['e', 'eades', 'eager']
F : ['f', 'faber', 'fabian']
G : ['g', 'ga', 'gaafer']
H : ['h', 'ha', 'haaek']
I : ['iberia', 'ibm', 'ibn']
J : ['j', 'ja', 'jab']
K : ['k', 'kabalevsky', 'kaboom']
L : ['l', 'la', 'lab']
M : ['mac', 'macabre', 'macarthur']
N : ['n', 'nab', 'nabbed']
O : ['oafs', 'oak', 'oaken']
P : ['p', 'pa', 'paba']
Q : ['q', 'qua', 'quack']
R : ['r', 'ra', 'rabat']
S : ['saabye', 'saadi', 'saami']
T : ['tab', 'tabac', 'tabb']
U : ['u', 'ubermenschen', 'ubiquitous']
V : ['v', 'va', 'vacancies']
W : ['w', 'wabash', 'wac']
X : ['x', 'xavier', 'xenia']
Y : ['ya', 'yacht', 'yachtel']
Z : ['z', 'zabel', 'zachrisson']
```

7. Expand any five casual slangs used in sentences into their corresponding formal forms

```python
slang_dict = {
    "uk": "you know",
    "idk": "I do not know",
    "brb": "be right back",
    "lol": "laughing out loud",
    "btw": "by the way"
}

sentences = [
    "idk what you mean uk",
    "brb I'll call you",
    "lol that was funny",
    "btw I finished the lab",
    "uk this is confusing"
]

def expand_slang(sentence):
    words = sentence.split()
    expanded = [slang_dict.get(w.lower(), w) for w in words]
    return " ".join(expanded)

print("\nSlang Expansion:")
for s in sentences:
    print("Original :", s)
    print("Expanded :", expand_slang(s))
    print()
```

```
...
Slang Expansion:
Original : idk what you mean uk
Expanded : I do not know what you mean you know

Original : brb I'll call you
Expanded : be right back I'll call you

Original : lol that was funny
Expanded : laughing out loud that was funny

Original : btw I finished the lab
Expanded : by the way I finished the lab

Original : uk this is confusing
Expanded : you know this is confusing
```

RESULT: Hence, the Brown corpus from the NLTK library was successfully imported and words were analysed.

EXP NO: **IMPLEMENTATION OF FINITE STATE AUTOMATA**

DATE:

AIM: To design a DFA that accepts binary strings which do not contain two consecutive 1s and to validate a string using a DFA transition table.

ALGORITHM:

DFA Without Two Consecutive 1s:

- Start
- Read the input binary string.
- Initialize state = 0 (start state).
- For each character ch in the input string:
  - If ch is not 0 or 1
    → Print "Invalid input" and terminate.
  - Else check current state:
    - If state = 0:
      - If ch = 0 → state = 0
      - If ch = 1 → state = 1
    - If state = 1:
      - If ch = 0 → state = 0
      - If ch = 1 → state = 2 (trap state)
    - If state = 2:
      - Stay in state 2 (trap state)
- After processing all characters:
  - If state = 0 or state = 1
    → Print "Accepted (No consecutive 1's)"
  - Else
    → Print "Rejected (Contains consecutive 1's)"
- Stop

DFA Using Transition Table:

- Start
- Read:
  - Number of states n
  - Number of input symbols m
- Create a transition table transition[n][m].
- For each state i from 0 to n-1:
  - For each symbol j from 0 to m-1:
    - Read δ(qi, j) and store in transition[i][j]
- Read the start state.
- Read number of final states f.
- Read the list of final states.
- Read the input string.
- Set currentState = startState.
- For each character ch in input string:
  - If ch is not a digit
    → Print "Invalid input symbol" and terminate.

- Convert character to integer:
  symbol = ch - '0'
- If symbol >= m
  → Print “Symbol not defined in DFA” and terminate.
- Update state:
  currentState = transition[currentState][symbol]

 After processing input:

- Check if currentState is in final states.
- If yes → Print “Accepted”
- Else → Print “Rejected”

 Stop

CODE:

DFA Without Two Consecutive 1s:

```
import java.util.Scanner;
public class DFA{
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter binary string: ");
      String input = sc.nextLine();
      int state = 0;
      for (int i = 0; i < input.length(); i++) {
         char ch = input.charAt(i);
         if (ch != '0' && ch != '1') {
            System.out.println("Invalid input! Only 0 and 1 allowed.");
            sc.close();
            return;
         }
         switch (state) {
            case 0:
               if (ch == '0')
                  state = 0;
               else
                  state = 1;
               break;
            case 1:
               if (ch == '0')
                  state = 0;
               else
```

```
                state = 2;
            break;
        case 2:
            state = 2;
            break;
        }
    }
    if (state == 0 || state == 1)
        System.out.println("Accepted (No consecutive 1's)");
    else
        System.out.println("Rejected (Contains consecutive 1's)");
    sc.close();
  }
}
```

DFA Using Transition Table:

```
import java.util.Scanner;

public class DFA_TT{

  public static void main(String[] args) {

    Scanner sc = new Scanner(System.in);

     System.out.print("Enter number of states: ");

    int n = sc.nextInt();

    System.out.print("Enter number of input symbols: ");

    int m = sc.nextInt();

    int[][] transition = new int[n][m];

    System.out.println("Enter transition table:");

    for (int i = 0; i < n; i++) {

        for (int j = 0; j < m; j++) {

            System.out.print("δ(q" + i + ", " + j + ") = ");

            transition[i][j] = sc.nextInt();
```

```
            }
        }
System.out.print("Enter start state: ");
        int startState = sc.nextInt();
        System.out.print("Enter number of final states: ");
        int f = sc.nextInt();
        int[] finalStates = new int[f];
        System.out.println("Enter final states:");
        for (int i = 0; i < f; i++) {
            finalStates[i] = sc.nextInt();
        }
        sc.nextLine();
        System.out.print("Enter input string: ");
        String input = sc.nextLine();
        int currentState = startState;
        for (int i = 0; i < input.length(); i++) {
            char ch = input.charAt(i);
            if (!Character.isDigit(ch)) {
                System.out.println("Invalid input symbol!");
                sc.close();
                return;
            }
            int symbol = ch - '0';
            if (symbol >= m) {
```

```
            System.out.println("Symbol not defined in DFA!");
            sc.close();
            return;
        }
        currentState = transition[currentState][symbol];
    }
    boolean accepted = false;
    for (int i = 0; i < f; i++) {
        if (currentState == finalStates[i]) {
            accepted = true;
            break;
        }
    }
    if (accepted)
        System.out.println("Result: Accepted");
    else
        System.out.println("Result: Rejected");
    sc.close();
  }
}
```

OUTPUT:

DFA Without Two Consecutive 1s:

```
it23a41@modern-computing-lab:~$ vi DFA.java
it23a41@modern-computing-lab:~$ javac DFA.java
it23a41@modern-computing-lab:~$ java DFA
Enter binary string: 10101
Accepted (No consecutive 1's)
it23a41@modern-computing-lab:~$ java DFA
Enter binary string: 110001
Rejected (Contains consecutive 1's)
it23a41@modern-computing-lab:~$
```

DFA Using Transition Table:

```
it23a41@modern-computing-lab:~$ vi DFA_TT.java
it23a41@modern-computing-lab:~$ javac DFA_TT.java
it23a41@modern-computing-lab:~$ java DFA_TT
Enter number of states: 3
Enter number of input symbols: 2
Enter transition table:
δ(q0, 0) = 0
δ(q0, 1) = 1
δ(q1, 0) = 0
δ(q1, 1) = 2
δ(q2, 0) = 2
δ(q2, 1) = 2
Enter start state: 0
Enter number of final states: 2
Enter final states:
0
1
Enter input string: 1010
Result: Accepted
it23a41@modern-computing-lab:~$ java DFA_TT
Enter number of states: 3
Enter number of input symbols: 2
Enter transition table:
δ(q0, 0) = 0
δ(q0, 1) = 1
δ(q1, 0) = 0
δ(q1, 1) = 2
δ(q2, 0) = 2
δ(q2, 1) = 2
Enter start state: 0
Enter number of final states: 2
Enter final states:
0
1
Enter input string: 1011
Result: Rejected
it23a41@modern-computing-lab:~$
```

RESULT: Hence, we designed a DFA that accepts binary strings which do not contain two consecutive 1s and validated a string using a DFA transition table.

EXP NO: **IMPLEMENTATION OF TEXT PREPROCESSING**

DATE:

AIM: To implement the fundamental text preprocessing techniques including filtration, script validation, tokenization, stop word removal and stemming.

ALGORITHM:

1. Import the required libraries and download necessary NLTK corpora.
2. Read the input string and convert it to lowercase.
3. Remove HTML tags from the text using regular expression filtration.
4. Perform script validation by removing all non-alphabetic characters.
5. Tokenize the cleaned text into individual words.
6. Count the frequency of each token and compute the number of unique words.
7. Remove stopwords from the tokens using the stopwords corpora.
8. Apply stemming to obtain the final processed words.

CODE & OUTPUT:

1. Download different corpora and import them.

```
import re
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer
from collections import Counter

nltk.download('punkt')
nltk.download('stopwords')
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
True
```

```
import nltk

nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('stopwords')
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
True
```

2. Get the input text.

```
text = input("Enter text: ")

Enter text: <html>This is an example of NLP preprocessing. The system removes stopwords and performs stemming.</html>
```

```
text = text.lower()
print("\nLowercase Text:")
print(text)

...
Lowercase Text:
<html>this is an example of nlp preprocessing. the system removes stopwords and performs stemming.</html>
```

3. Filter the script (remove HTML tags).

```
text = re.sub(r'<.*?>', '', text)

print("\nAfter removing HTML tags:")
print(text)

After removing HTML tags:
this is an example of nlp preprocessing. the system removes stopwords and performs stemming.
```

4. Validate the script (must contain only alphabets).

```
text = re.sub(r'[^a-z\s]', '', text)

print("\nAfter removing non-alphabet characters:")
print(text)

...
After removing non-alphabet characters:
this is an example of nlp preprocessing the system removes stopwords and performs stemming
```

5. Tokenize the text script.

```
tokens = word_tokenize(text)

print("\nTokens:")
print(tokens)

Tokens:
['this', 'is', 'an', 'example', 'of', 'nlp', 'preprocessing', 'the', 'system', 'removes', 'stopwords', 'and', 'performs', 'stemming']
```

```
freq = Counter(tokens)

print("\nToken Frequency:")
print(freq)

...
Token Frequency:
Counter({'this': 1, 'is': 1, 'an': 1, 'example': 1, 'of': 1, 'nlp': 1, 'preprocessing': 1, 'the': 1, 'system': 1, 'removes': 1, 'stopwords': 1, 'and': 1, 'performs': 1, 'stemming': 1})
```

6. Remove stop words from the text script.

```
stop_words = set(stopwords.words('english'))

filtered_tokens = [word for word in tokens if word not in stop_words]

print("\nAfter Stopword Removal:")
print(filtered_tokens)
```

```
After Stopword Removal:
['example', 'nlp', 'preprocessing', 'system', 'removes', 'stopwords', 'performs', 'stemming']
```

7. Apply stemming over the script.

```
stemmer = PorterStemmer()

stemmed_words = [stemmer.stem(word) for word in filtered_tokens]

print("\nAfter Stemming:")
print(stemmed_words)
```

```
After Stemming:
['exampl', 'nlp', 'preprocess', 'system', 'remov', 'stopword', 'perform', 'stem']
```

RESULT: Hence, various text preprocessing techniques have been implemented successfully.

EXP NO:	**IMPLEMENTATION OF WORD GENERATION**

DATE:

AIM: To perform word generation using NLTK.

ALGORITHM:

1. Import required NLTK libraries and download necessary corpora.
2. Create a set of English dictionary words for fast lookup.
3. Generate 10 random words by sampling from the dictionary list.
4. Take user input and check whether the word exists in the dictionary set.
5. Generate all letter combinations of length K from a given word and print those present in the dictionary.
6. Extract 100 random words from the Brown corpus after filtering alphabetic words.
7. Add predefined prefixes and suffixes to those words and check if the new words are valid dictionary words.
8. Remove stop words and punctuation from Brown corpus words using a loop and store unique words.
9. Calculate the average length of the unique cleaned words using a simple loop.

CODE & OUTPUT:

1. Download different corpora and import them.

```
import nltk
import random
import string
from itertools import combinations

nltk.download('words')
nltk.download('brown')
nltk.download('stopwords')

from nltk.corpus import words, brown, stopwords
```

```
[nltk_data] Downloading package words to /root/nltk_data...
[nltk_data]   Unzipping corpora/words.zip.
[nltk_data] Downloading package brown to /root/nltk_data...
[nltk_data]   Unzipping corpora/brown.zip.
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
```

2. Generate 10 random words from the English dictionary.

```
dictionary_words = set(words.words())

random_words = random.sample(list(dictionary_words), 10)

print("10 Random Words from Dictionary:")
print(random_words)
```

```
10 Random Words from Dictionary:
['horseweed', 'forbore', 'unforeseeable', 'madwort', 'inditer', 'rammelsbergite', 'flu', 'thought', 'untranspired', 'delphinoid']
```

3. Create a utility function to get input and check if the given word is valid in the given dictionary.

```
def check_word():
    user_word = input("Enter a word: ").lower()

    if user_word in dictionary_words:
        print("The word exists in the dictionary.")
    else:
        print("The word does NOT exist in the dictionary.")

check_word()
```

```
Enter a word: Hello
The word exists in the dictionary.
```

```
def check_word():
    user_word = input("Enter a word: ").lower()

    if user_word in dictionary_words:
        print("The word exists in the dictionary.")
    else:
        print("The word does NOT exist in the dictionary.")

check_word()
```

```
Enter a word: r#
The word does NOT exist in the dictionary.
```

4. Write a function that takes a string and an integer K as input, and print all valid letter combination from the given word of the length K.

```python
def valid_combinations(word, k):
    word = word.lower()
    comb = combinations(word, k)

    valid_words = []

    for c in comb:
        new_word = ''.join(c)
        if new_word in dictionary_words:
            valid_words.append(new_word)

    print("Valid combinations:", valid_words)

valid_combinations("plant", 3)
```

```
Valid combinations: ['pan', 'pat', 'lan', 'lat', 'ant']
```

5. Generate 100 new words from the brown corpus. Select a word and add prefix and suffix and check whether it is a valid word. Print all such valid words.

```python
brown_words = [w.lower() for w in brown.words() if w.isalpha()]

sample_words = random.sample(brown_words, 100)

prefixes = ["un", "re", "pre"]
suffixes = ["ing", "ed", "ly"]

valid_new_words = []

for word in sample_words:
    for p in prefixes:
        new_word = p + word
        if new_word in dictionary_words:
            valid_new_words.append(new_word)

    for s in suffixes:
        new_word = word + s
        if new_word in dictionary_words:
            valid_new_words.append(new_word)

print("Valid new words with prefix/suffix:")
print(valid_new_words)
```

```
Valid new words with prefix/suffix:
['rethe', 'unload', 'reload', 'loading', 'loaded', 'unto', 'toed', 'rein'
```

6. Extract unique words from brown corpus and remove stopwords, punctuations in a list. Also calculate the average word length.

```python
stop_words = set(stopwords.words('english'))

clean_words = []

for word in brown.words():
    word = word.lower()

    if word.isalpha() and word not in stop_words:
        clean_words.append(word)

unique_words = set(clean_words)

print("Number of unique cleaned words:", len(unique_words))

total_length = 0

for word in unique_words:
    total_length += len(word)

avg_length = total_length / len(unique_words)

print("Average word length:", avg_length)
```

```
Number of unique cleaned words: 40097
Average word length: 7.821956754869442
```

RESULT: Hence, word generation has been performed successfully.

EXP NO: **IMPLEMENTATION OF MORPHOLOGICAL ANALYSIS**
DATE:

AIM: To perform morphological analysis using python.

ALGORITHM:

1. Start the program and load the spaCy English language model.
2. Read a sentence as input from the user.
3. Process the input sentence using spaCy NLP pipeline to obtain tokens and linguistic features.
4. For each token in the sentence, extract its lemma, part-of-speech (POS), and morphological features such as person, tense, and number.
5. Format the extracted information into a structured table format.
6. Display the word, lemma, POS tag, and morphological features for each token.

CODE:

1. Extract the original words, its lemma, it's part of speech (POS), its morphological features [Person (1/2/3), tense(present/past/future) and number(singular/plural)].

OUTPUT:

```python
import spacy

nlp = spacy.load("en_core_web_sm")
```

```python
sentence = input("Enter a sentence: ")
```

```
Enter a sentence: She writes letters and they were delivered yesterday
```

```python
doc = nlp(sentence)
```

```python
print("{:<15} {:<15} {:<10} {:<10} {:<10} {:<10}".format(
    "Word", "Lemma", "POS", "Person", "Tense", "Number"
))
print("-"*70)

for token in doc:

    person = token.morph.get("Person")
    tense = token.morph.get("Tense")
    number = token.morph.get("Number")


    person = person[0] if person else "-"
    tense = tense[0] if tense else "-"
    number = number[0] if number else "-"

    print("{:<15} {:<15} {:<10} {:<10} {:<10} {:<10}".format(
        token.text,
        token.lemma_,
        token.pos_,
        person,
        tense,
        number
    ))
```

```
Word            Lemma           POS        Person     Tense      Number
----------------------------------------------------------------------
She             she             PRON       3          -          Sing
writes          write           VERB       3          Pres       Sing
letters         letter          NOUN       -          -          Plur
and             and             CCONJ      -          -          -
they            they            PRON       3          -          Plur
were            be              AUX        -          Past       -
delivered       deliver         VERB       -          Past       -
yesterday       yesterday       NOUN       -          -          Sing
```

RESULT: Hence, morphological analysis has been performed successfully.

EXP NO: **IMPLEMENTATION OF N-GRAM ANALYSIS**
DATE:

AIM: To perform N-Gram analysis using python.

ALGORITHM:

1. Start.
2. Initialize Resources:
    - Import necessary libraries.
    - Download required datasets: brown, treebank, and the punkt tokenizer.
3. Data Preprocessing (Model Building):
    - Load all words from the combined Brown and Treebank corpora.
    - Convert all words to **lowercase** to ensure uniformity.
    - Generate and store counts for:
        - Unigrams: Individual word frequencies.
        - Bigrams: Pairs of consecutive words.
        - Trigrams: Sequences of three consecutive words.
4. User Input: Accept a string or phrase from the user.
5. Tokenization: Convert the user input into a list of lowercase tokens.
6. Bigram Analysis:
    - Identify the last word of the input.
    - Search the bigram model for all pairs starting with that word.
    - Calculate the **probability** for each potential next word.
    - Display the top 10 most frequent following words.
7. Trigram Analysis:
    - Identify the last two words of the input.
    - Search the trigram model for sequences starting with those two words.
    - Calculate the **probability** for each potential next word.
    - Display the top 3 most frequent following words.
8. Repeat: Continue accepting input until the user types 'exit'.
9. Stop.

QUESTION:

1. Write a program to implement N-Gram Analysis using the NLTK library. The program should load standard text corpora (Brown and Treebank), build unigram, bigram, and trigram models, and predict the most probable next word based on a user-provided input string by calculating frequencies and probabilities.

CODE:

```
import nltk
from nltk.corpus import brown, treebank
from nltk.util import ngrams
from collections import Counter

nltk.download('brown')
```

```
nltk.download('treebank')
nltk.download('punkt')

def build_ngram_models():
    print("Loading and processing corpora...")
    words = [w.lower() for w in brown.words()] + [w.lower() for w in treebank.words()]

    unigram_counts = Counter(words)
    bigram_counts = Counter(ngrams(words, 2))
    trigram_counts = Counter(ngrams(words, 3))

    return unigram_counts, bigram_counts, trigram_counts

def analyze_input(user_input, unigrams, bigrams, trigrams):
    tokens = nltk.word_tokenize(user_input.lower())
    if not tokens:
        return

    last_word = tokens[-1]
    print(f"\n--- Bigram Analysis for word after '{last_word}' ---")

    possible_next_bigrams = {bg[1]: count for bg, count in bigrams.items() if bg[0] ==
last_word}

    if possible_next_bigrams:
        top_next = sorted(possible_next_bigrams.items(), key=lambda x: x[1],
reverse=True)[:10]
        for word, freq in top_next:
            prob = freq / unigrams[last_word]
            print(f"Word: '{word}' | Frequency: {freq} | Probability: {prob:.5f}")
    else:
        print("No bigram matches found in corpus.")

    if len(tokens) >= 2:
        last_two = tuple(tokens[-2:])
        print(f"\n--- Trigram Analysis for word after '{' '.join(last_two)}' ---")

        possible_next_trigrams = {tg[2]: count for tg, count in trigrams.items() if tg[:2] ==
last_two}

        if possible_next_trigrams:
            top_next = sorted(possible_next_trigrams.items(), key=lambda x: x[1],
reverse=True)[:3]
            for word, freq in top_next:
                prob = freq / bigrams[last_two]
                print(f"Word: '{word}' | Frequency: {freq} | Probability: {prob:.5f}")
        else:
            print("No trigram matches found in corpus.")

if __name__ == "__main__":
```

```
u_counts, b_counts, t_counts = build_ngram_models()

while True:
    user_text = input("\nEnter 1 or more words (or 'exit'): ")
    if user_text.lower() == 'exit':
        break
    analyze_input(user_text, u_counts, b_counts, t_counts)
```

OUTPUT:

```
Loading and processing corpora...

Enter 1 or more words (or 'exit'):  I am

--- Bigram Analysis for word after 'am' ---
Word: 'not' | Frequency: 25 | Probability: 0.10460
Word: 'i' | Frequency: 14 | Probability: 0.05858
Word: 'a' | Frequency: 13 | Probability: 0.05439
Word: 'sure' | Frequency: 10 | Probability: 0.04184
Word: '.' | Frequency: 7 | Probability: 0.02929
Word: '''' | Frequency: 5 | Probability: 0.02092
Word: 'an' | Frequency: 5 | Probability: 0.02092
Word: 'innocent' | Frequency: 5 | Probability: 0.02092
Word: 'told' | Frequency: 4 | Probability: 0.01674
Word: 'very' | Frequency: 4 | Probability: 0.01674

--- Trigram Analysis for word after 'i am' ---
Word: 'not' | Frequency: 24 | Probability: 0.11538
Word: 'a' | Frequency: 13 | Probability: 0.06250
Word: 'sure' | Frequency: 10 | Probability: 0.04808


Enter 1 or more words (or 'exit'):  I am in

--- Bigram Analysis for word after 'in' ---
Word: 'the' | Frequency: 6415 | Probability: 0.27763
Word: 'a' | Frequency: 1516 | Probability: 0.06561
Word: 'his' | Frequency: 631 | Probability: 0.02731
Word: 'this' | Frequency: 554 | Probability: 0.02398
Word: 'which' | Frequency: 388 | Probability: 0.01679
Word: 'their' | Frequency: 264 | Probability: 0.01143
Word: 'an' | Frequency: 259 | Probability: 0.01121
Word: 'its' | Frequency: 194 | Probability: 0.00840
Word: 'fact' | Frequency: 159 | Probability: 0.00688
Word: 'her' | Frequency: 159 | Probability: 0.00688

--- Trigram Analysis for word after 'am in' ---
Word: 'great' | Frequency: 1 | Probability: 0.33333
Word: 'good' | Frequency: 1 | Probability: 0.33333
Word: 'it' | Frequency: 1 | Probability: 0.33333

Enter 1 or more words (or 'exit'):  exit
```

2. Construct a Bigram Language Model using a corpus to generate vocabulary, compute unigram frequencies, calculate bigram probabilities with and without Laplace smoothing, create a bigram probability matrix, and evaluate sentences using sentence probability and perplexity score.

CODE:

```
import nltk

from nltk.corpus import brown

from nltk.util import ngrams

from collections import Counter

import pandas as pd

import math


nltk.download('brown')

nltk.download('punkt')

nltk.download('punkt_tab')


words = [w.lower() for w in brown.words()]


vocab = sorted(set(words))

V = len(vocab)

print("Vocabulary Size:", V)


unigram_counts = Counter(words)

bigrams = list(ngrams(words, 2))

bigram_counts = Counter(bigrams)


print("Sample Vocabulary:", vocab[:10])
```

```
print("Sample Unigrams:", list(unigram_counts.items())[:10])
print("Sample Bigrams:", bigrams[:10])

def bigram_prob(w1, w2):
    if unigram_counts[w1] == 0:
        return 0
    return bigram_counts[(w1, w2)] / unigram_counts[w1]

def bigram_prob_laplace(w1, w2):
    return (bigram_counts[(w1, w2)] + 1) / (unigram_counts[w1] + V)

subset_vocab = vocab[:30]
bigram_table = pd.DataFrame(index=subset_vocab, columns=subset_vocab)

for w1 in subset_vocab:
    for w2 in subset_vocab:
        bigram_table.loc[w1, w2] = bigram_prob(w1, w2)

bigram_table.fillna(0, inplace=True)
print("\nBigram Probability Table (No Smoothing):")
print(bigram_table.head())

bigram_table_smooth = pd.DataFrame(index=subset_vocab, columns=subset_vocab)

for w1 in subset_vocab:
```

```
    for w2 in subset_vocab:
        bigram_table_smooth.loc[w1, w2] = bigram_prob_laplace(w1, w2)

print("\nBigram Probability Table (With Laplace Smoothing):")
print(bigram_table_smooth.head())

def sentence_probability(sentence, smooth=False):
    tokens = nltk.word_tokenize(sentence.lower())
    prob = 1
    for w1, w2 in ngrams(tokens, 2):
        if smooth:
            prob *= bigram_prob_laplace(w1, w2)
        else:
            prob *= bigram_prob(w1, w2)
    return prob

def perplexity(sentence):
    tokens = nltk.word_tokenize(sentence.lower())
    N = len(tokens)
    prob = sentence_probability(sentence, smooth=True)
    return pow(1/prob, 1/N)

test_sentence = input("\nEnter a sentence: ")
print("\nSentence:", test_sentence)
print("Sentence Probability (No smoothing):", sentence_probability(test_sentence))
```

print("Sentence Probability (Laplace):", sentence_probability(test_sentence, smooth=True))

print("Perplexity:", perplexity(test_sentence))

OUTPUT:

```
Vocabulary Size: 49815
Sample Vocabulary: ['!', '$.027', '$.03', '$.054/mbf', '$.07', '$.07/cwt', '$.076
Sample Unigrams: [('the', 69971), ('fulton', 17), ('county', 155), ('grand', 48),
Sample Bigrams: [('the', 'fulton'), ('fulton', 'county'), ('county', 'grand'), ('

Bigram Probability Table (No Smoothing):
             ! $.027 $.03 $.054/mbf $.07 $.07/cwt $.076 $.09 $.10-a-minute  \
!          0.5   0.0  0.0       0.0  0.0      0.0   0.0  0.0           0.0
$.027      0.0   0.0  0.0       0.0  0.0      0.0   0.0  0.0           0.0
$.03       0.0   0.0  0.0       0.0  0.0      0.0   0.0  0.0           0.0
$.054/mbf  0.0   0.0  0.0       0.0  0.0      0.0   0.0  0.0           0.0
$.07       0.0   0.0  0.0       0.0  0.0      0.0   0.0  0.0           0.0

          $.105  ...   $1 $1,000 $1,000,000 $1,000,000,000 $1,200 $1,250,000  \
!           0.0  ...  0.0    0.0        0.0            0.0    0.0        0.0
$.027       0.0  ...  0.0    0.0        0.0            0.0    0.0        0.0
$.03        0.0  ...  0.0    0.0        0.0            0.0    0.0        0.0
$.054/mbf   0.0  ...  0.0    0.0        0.0            0.0    0.0        0.0
$.07        0.0  ...  0.0    0.0        0.0            0.0    0.0        0.0

          $1,276 $1,390 $1,450,000,000 $1,500
!            0.0    0.0            0.0    0.0
$.027        0.0    0.0            0.0    0.0
$.03         0.0    0.0            0.0    0.0
$.054/mbf    0.0    0.0            0.0    0.0
$.07         0.0    0.0            0.0    0.0

[5 rows x 30 columns]

Bigram Probability Table (With Laplace Smoothing):
                  !     $.027      $.03 $.054/mbf      $.07  $.07/cwt  \
!          0.015541  0.000019  0.000019  0.000019  0.000019  0.000019
$.027       0.00002   0.00002   0.00002   0.00002   0.00002   0.00002
$.03        0.00002   0.00002   0.00002   0.00002   0.00002   0.00002
$.054/mbf   0.00002   0.00002   0.00002   0.00002   0.00002   0.00002
$.07        0.00002   0.00002   0.00002   0.00002   0.00002   0.00002

              $.076      $.09 $.10-a-minute     $.105  ...        $1  \
!          0.000019  0.000019      0.000019  0.000019  ...  0.000019
$.027       0.00002   0.00002       0.00002   0.00002  ...   0.00002
$.03        0.00002   0.00002       0.00002   0.00002  ...   0.00002
$.054/mbf   0.00002   0.00002       0.00002   0.00002  ...   0.00002
$.07        0.00002   0.00002       0.00002   0.00002  ...   0.00002

             $1,000 $1,000,000 $1,000,000,000    $1,200 $1,250,000    $1,276  \
!          0.000019   0.000019       0.000019  0.000019   0.000019  0.000019
$.027       0.00002    0.00002        0.00002   0.00002    0.00002   0.00002
$.03        0.00002    0.00002        0.00002   0.00002    0.00002   0.00002
$.054/mbf   0.00002    0.00002        0.00002   0.00002    0.00002   0.00002
$.07        0.00002    0.00002        0.00002   0.00002    0.00002   0.00002
```

```
             $1,390 $1,450,000,000    $1,500
!          0.000019       0.000019  0.000019
$.027       0.00002        0.00002   0.00002
$.03        0.00002        0.00002   0.00002
$.054/mbf   0.00002        0.00002   0.00002
$.07        0.00002        0.00002   0.00002

[5 rows x 30 columns]

Enter a sentence: the jury said the county

Sentence: the jury said the county
Sentence Probability (No smoothing): 7.620149391332032e-10
Sentence Probability (Laplace): 1.6174551957917738e-14
Perplexity: 573.1041431593538
```

RESULT: Hence, N-Gram analysis has been performed successfully.

EXP NO: 8 **IMPLEMENTATION OF POS TAGGING USING HMM**
DATE:

AIM: To implement Parts of Speech (POS) tagging using a Hidden Markov Model (HMM) by computing transition and emission probabilities from a tagged corpus.

ALGORITHM:

- Load a tagged corpus.
- Extract words and their POS tags.
- Compute:
  1. **Transition Probability**: $P(t_i \mid t_{i-1})$
  2. **Emission Probability**: $P(w_i \mid t_i)$

- Store these probabilities.
- Use them to understand how tags and words relate.

CODE:

```
import nltk

from nltk.corpus import treebank

from collections import defaultdict, Counter



nltk.download('treebank')



tagged_sentences = treebank.tagged_sents()



transition_counts = defaultdict(Counter)

emission_counts = defaultdict(Counter)

tag_counts = Counter()



for sentence in tagged_sentences:

    prev_tag = None

    for word, tag in sentence:
```

```
        tag_counts[tag] += 1
        emission_counts[tag][word.lower()] += 1
        if prev_tag is not None:
            transition_counts[prev_tag][tag] += 1
        prev_tag = tag

transition_prob = defaultdict(dict)
emission_prob = defaultdict(dict)

for prev_tag in transition_counts:
    total = sum(transition_counts[prev_tag].values())
    for tag in transition_counts[prev_tag]:
        transition_prob[prev_tag][tag] = transition_counts[prev_tag][tag] / total

for tag in emission_counts:
    total = sum(emission_counts[tag].values())
    for word in emission_counts[tag]:
        emission_prob[tag][word] = emission_counts[tag][word] / total

print("Sample Transition Probability:", list(transition_prob.items())[:5])
print("Sample Emission Probability:", list(emission_prob.items())[:5])

def hmm_greedy_tagging(sentence):
    words = sentence.lower().split()
    tags_list = list(tag_counts.keys())
```

```
    predicted_tags = []
    prev_tag = None

    for word in words:
        best_tag = None
        max_prob = 0

        for tag in tags_list:
            emis = emission_prob[tag].get(word, 1e-6)

            if prev_tag is None:
                prob = emis
            else:
                trans = transition_prob[prev_tag].get(tag, 1e-6)
                prob = emis * trans

            if prob > max_prob:
                max_prob = prob
                best_tag = tag

        predicted_tags.append((word, best_tag))
        prev_tag = best_tag

    return predicted_tags
```

```
sentence = input("Enter a sentence for HMM tagging: ")

print(hmm_greedy_tagging(sentence))
```

OUTPUT:

```
[nltk_data] Downloading package treebank to /root/nltk_data...
[nltk_data]   Package treebank is already up-to-date!
Sample Transition Probability: [('NNP', {'NNP': 0.3825374880357333, ',': 0.15324896309688396, 'CD': 0.
Sample Emission Probability: [('NNP', {'pierre': 0.00010626992561105207, 'vinken': 0.00021253985122210
Enter a sentence for HMM tagging: Hi this is John
[('hi', 'NNP'), ('this', 'DT'), ('is', 'VBZ'), ('john', 'NNP')]
```

RESULT: Hence, POS tagging using HMM has been performed successfully.

EXP: 9 IMPLEMENTATION OF POS TAGGING USING VITERBI ALGORITHM
DATE:

AIM: To implement the Viterbi algorithm to determine the most probable sequence of POS tags for a given sentence using HMM probabilities.

ALGORITHM:

- Take a sentence as input.
- Initialize probabilities using emission and transition probabilities.
- For each word, compute the most probable tag path.
- Backtrack to find the best tag sequence.

CODE:

```
import math


tags = list(tag_counts.keys())


def viterbi(sentence):

    words = sentence.lower().split()

    V = [{}]

    path = {}



    for tag in tags:

        emis = emission_prob[tag].get(words[0], 1e-6)

        V[0][tag] = emis

        path[tag] = [tag]


    for t in range(1, len(words)):

        V.append({})
```

```
        new_path = {}

        for curr_tag in tags:
            (prob, prev_tag) = max(
                (V[t-1][pt] *
                 transition_prob[pt].get(curr_tag, 1e-6) *
                 emission_prob[curr_tag].get(words[t], 1e-6), pt)
                for pt in tags
            )

            V[t][curr_tag] = prob
            new_path[curr_tag] = path[prev_tag] + [curr_tag]

        path = new_path

    n = len(words) - 1
    (prob, tag) = max((V[n][t], t) for t in tags)
    return list(zip(words, path[tag]))

sentence = input("Enter a sentence: ")
print(viterbi(sentence))
```

OUTPUT:

```
Enter a sentence: Hello this is John
[('hello', 'PDT'), ('this', 'DT'), ('is', 'VBZ'), ('john', 'NNP')]
```

RESULT: Hence, POS tagging using Viterbi Algorithm has been performed successfully.

EXP NO: IMPLEMENTATION OF MINIMUM EDIT DISTANCE ALGORITHM
DATE:

AIM: To compute the minimum edit distance between two strings by constructing the edit distance table and determining the minimum number of operations (insert, delete, substitute) required to convert one string into another.

ALGORITHM:

1. Start.
2. Read two input strings: source and target.
3. Let length of source = m, target = n.
4. Create a matrix dp[m+1][n+1].
5. Initialize:
   - First row: dp[0][j] = j
   - First column: dp[i][0] = i
6. For each cell dp[i][j]:
   - If source[i-1] == target[j-1],
     dp[i][j] = dp[i-1][j-1]
   - Else
     dp[i][j] = 1 + min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1])
     - delete
     - insert
     - substitute
7. Display the matrix.
8. Minimum edit distance = dp[m][n].
9. Stop.

CODE:

```
import java.util.Scanner;

public class MinimumEditDistance {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter source string: ");
        String source = sc.nextLine();

        System.out.print("Enter target string: ");
        String target = sc.nextLine();

        int m = source.length();
        int n = target.length();
```

```
        int[][] dp = new int[m + 1][n + 1];


        for (int i = 0; i <= m; i++) {
            dp[i][0] = i;
        }

        for (int j = 0; j <= n; j++) {
            dp[0][j] = j;
        }

        for (int i = 1; i <= m; i++) {
            for (int j = 1; j <= n; j++) {

                if (source.charAt(i - 1) == target.charAt(j - 1)) {
                    dp[i][j] = dp[i - 1][j - 1];
                } else {
                    dp[i][j] = 1 + Math.min(
                            dp[i - 1][j],
                            Math.min(
                                    dp[i][j - 1],
                                    dp[i - 1][j - 1]
                            )
                    );
                }
            }
        }

        System.out.println("\nEdit Distance Table:");
        for (int i = 0; i <= m; i++) {
            for (int j = 0; j <= n; j++) {
                System.out.print(dp[i][j] + " ");
            }
            System.out.println();
        }

        System.out.println("\nMinimum Edit Distance: " + dp[m][n]);
    }
}
```

OUTPUT:

```
it23a41@nlpds-lab:~$ vi MinimumEditDistance.java
it23a41@nlpds-lab:~$ javac MinimumEditDistance.java
it23a41@nlpds-lab:~$ java MinimumEditDistance
Enter source string: TWISTER
Enter target string: SISTER

Edit Distance Table:
0 1 2 3 4 5 6
1 1 2 3 3 4 5
2 2 2 3 4 4 5
3 3 2 3 4 5 5
4 3 3 2 3 4 5
5 4 4 3 2 3 4
6 5 5 4 3 2 3
7 6 6 5 4 3 2

Minimum Edit Distance: 2
it23a41@nlpds-lab:~$
```

RESULT: Hence, the Minimum Edit Distance Algorithm has been implemented successfully.

EXP NO: **IMPLEMENTATION OF BUILDING A POS TAGGER**
DATE:

AIM: To build a Part-of-Speech (POS) tagger using a Unigram model trained on the Brown Corpus via Natural Language Toolkit (NLTK) and evaluate its tagging accuracy.

ALGORITHM:

- Import required libraries from NLTK.
- Download and load the tagged sentences from the Brown corpus (news category).
- Split the dataset into:
  1. 90% training data
  2. 10% testing data
- Train a UnigramTagger on the training sentences.
- Evaluate the tagger's accuracy on the test sentences.
- Accept a user sentence as input.
- Tag each word in the sentence with its predicted POS tag.

CODE:

```
import nltk

from nltk.corpus import brown

from nltk.tag import UnigramTagger



nltk.download('brown')



brown_tagged_sents = brown.tagged_sents(categories='news')

size = int(len(brown_tagged_sents) * 0.9)

train_sents = brown_tagged_sents[:size]

test_sents = brown_tagged_sents[size:]



unigram_tagger = UnigramTagger(train_sents)
```

```
accuracy = unigram_tagger.accuracy(test_sents)

print(f"Tagger Accuracy: {accuracy:.2%}")


sentence = input("Enter a sentence: ").split()

print(unigram_tagger.tag(sentence))
```

OUTPUT:

```
[nltk_data] Downloading package brown to /home/it23a41/nltk_data...
[nltk_data]   Unzipping corpora/brown.zip.
Tagger Accuracy: 81.21%
Enter a sentence:  The jury said it was a success
[('The', 'AT'), ('jury', 'NN'), ('said', 'VBD'), ('it', 'PPS'), ('was', 'BEDZ'), ('a', 'AT'), ('success', 'NN')]
```

RESULT: Hence, a POS Tagger has been built successfully.

EXP NO: IMPLEMENTATION OF CHUNKING TO EXTRACT NOUN PHRASES
DATE:

AIM: To implement chunking for extracting Noun Phrases (NPs) from a sentence using spaCy and its English model en_core_web_sm.

ALGORITHM:

- Import the spaCy library.
- Load the pre-trained English language model en_core_web_sm.
- Accept a sentence as input from the user.
- Process the sentence using the spaCy NLP pipeline.
- Iterate through doc.noun_chunks.
- Print each noun phrase and its root word.

CODE:

```
import spacy


nlp = spacy.load("en_core_web_sm")


sentence = input("Enter a sentence: ")

doc = nlp(sentence)


for chunk in doc.noun_chunks:

    print(f"Text: {chunk.text} | Root: {chunk.root.text}")
```

OUTPUT:

```
Enter a sentence:  Hi this is John
Text: this | Root: this
Text: John | Root: John
```

```
Enter a sentence:  The quick brown fox jumps over the lazy dog
Text: The quick brown fox | Root: fox
Text: the lazy dog | Root: dog
```

RESULT: Hence, chunking for extracting Noun Phrases has been implemented successfully.

EXP NO: **IMPLEMENTATION OF BUILDING A CHUNKER**
DATE:

AIM: To build a rule-based noun phrase chunker using regular expression grammar with Natural Language Toolkit (NLTK)**.**

ALGORITHM:

- Import necessary modules from NLTK.
- Accept a sentence as input.
- Tokenize the sentence into words.
- Perform POS tagging on the tokens.
- Define a chunk grammar using regular expressions:
  1. Optional Determiner <DT>?
  2. Zero or more Adjectives <JJ>*
  3. A Noun <NN>

- Create a RegexpParser with the grammar.
- Parse the tagged sentence to form chunks.
- Display the chunk tree.

CODE:

```
import nltk

from nltk import pos_tag, word_tokenize, RegexpParser



text = input("Enter a sentence: ")



tokens = word_tokenize(text)

tagged = pos_tag(tokens)



grammar = r"""

  NP: {<DT>?<JJ>*<NN|NNS|NNP|NNPS>}

"""
```

```
cp = RegexpParser(grammar)

result = cp.parse(tagged)


print(result)
```

OUTPUT:

```
Enter a sentence:  The quick brown fox jumps over the lazy dog
(S
  (NP The/DT quick/JJ brown/NN)
  (NP fox/NN)
  jumps/VBZ
  over/IN
  (NP the/DT lazy/JJ dog/NN))
```

RESULT: Hence, a Chunker has been built successfully.

EXP NO: IMPLEMENTATION OF NAME ENTITY RECOGNITION ON TEXT
DATE:

AIM: To perform Named Entity Recognition on text using Natural Language Toolkit (NLTK) and identify entities such as persons, organizations, and locations.

ALGORITHM:

1. Import required modules from NLTK.
2. Download necessary NLTK resources:
    - Tokenizer
    - POS tagger
    - Named entity chunker
    - Word corpus
3. Accept a sentence as input.
4. Tokenize the sentence into words.
5. Perform POS tagging on the tokens.
6. Apply ne_chunk() to identify named entities.
7. Display the chunk tree showing entities.

CODE:

```
import nltk

from nltk import word_tokenize, pos_tag, ne_chunk



nltk.download('punkt')

nltk.download('maxent_ne_chunker')

nltk.download('words')

nltk.download('averaged_perceptron_tagger')



text = input("Enter a sentence: ")



tokens = word_tokenize(text)

pos_tags = pos_tag(tokens)
```

```
entities = ne_chunk(pos_tags)

print(entities)
```

OUTPUT:

```
Enter a sentence:  Steve Jobs founded Apple in California.
(S
  (PERSON Steve/NNP)
  (PERSON Jobs/NNP)
  founded/VBD
  Apple/NNP
  in/IN
  (GPE California/NNP)
  ./.)
```

RESULT: Hence, Named Entity Recognition on text has been implemented successfully.

EXP NO: **KOREAN SONG LYRICS TRANSLATOR WITH INTERACTIVE**
DATE: **VOCABULARY FLASHCARDS AND PRONUNCIATION**

AIM: To design and implement a real-time NLP application that:

- Translates Korean song lyrics to English,
- Extracts meaningful Korean vocabulary,
- Presents them as interactive flashcards with pronunciation and audio,
- And evaluates the system using standard statistical metrics.

Tools, Models, and Resources:

- Translation model from Helsinki-NLP hosted on Hugging Face
- Korean morphological analysis using KoNLPy
- Romanization using hangul-romanize
- Text-to-Speech using gTTS
- Interactive web interface using Streamlit
- Evaluation using BLEU metric via sacrebleu

Summary:

This mini-project implements a complete NLP pipeline around Korean song lyrics:

1. **User Input**: Korean lyrics are entered through a web interface.
2. **Machine Translation**: Lyrics are translated to English using a transformer-based model.
3. **Word Extraction**: Important Korean words (nouns, verbs, adjectives) are extracted using morphological POS analysis.
4. **Flashcard Generation**: For each extracted word, the system shows:
    - Korean word
    - Romanized pronunciation
    - English meaning
    - Audio pronunciation
5. **Interactive UI**: Flashcards can be navigated (Next/Previous) similar to Duolingo.
6. **Performance Evaluation**:
    - Translation quality measured using BLEU score.
    - Vocabulary extraction evaluated using precision.
7. **Real-time Validation**:
    - Lyrics from a Korean song was tested.
    - The English translation produced by the system was observed to be very close to results from Google Translate, validating practical correctness.

Findings / Results:

Component	Metric	Observed Value	Interpretation
Translation	BLEU Score	**18.35**	Moderate BLEU due to lyrical/poetic text differing from formal training data, while meaning preserved
Word Extraction	Precision	**50%**	Extractor returns all valid content words; evaluation limited by small manual reference set
Real-time Test	Manual comparison	Close to Google Translate	Confirms practical semantic accuracy

Key Observations:

- The translation model preserves meaning even when wording differs (BLEU limitation).
- Korean POS-based extraction successfully identifies learning-worthy vocabulary.
- Audio + romanization significantly enhance usability as a learning tool.
- The app behaves like a real educational product, not just a script.

CODE AND OUTPUT:

Korean Song Lyrics Translator with Vocabulary Flashcards and Pronunciation Support

Install libraries

```
!pip install transformers sentencepiece konlpy hangul-romanize gtts
```

```
Requirement already satisfied: transformers in /usr/local/lib/python3.12/dist-packages (5.0.0)
Requirement already satisfied: sentencepiece in /usr/local/lib/python3.12/dist-packages (0.2.1)
Requirement already satisfied: konlpy in /usr/local/lib/python3.12/dist-packages (0.6.0)
Requirement already satisfied: hangul-romanize in /usr/local/lib/python3.12/dist-packages (0.1.0)
Requirement already satisfied: gtts in /usr/local/lib/python3.12/dist-packages (2.5.4)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from transformers) (3.25.2)
Requirement already satisfied: huggingface-hub<2.0,>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (1.10.1)
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist-packages (from transformers) (2.0.2)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (26.0)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from transformers) (6.0.3)
Requirement already satisfied: regex!=2019.12.17 in /usr/local/lib/python3.12/dist-packages (from transformers) (2025.11.3)
Requirement already satisfied: tokenizers<=0.23.0,>=0.22.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (0.22.2)
Requirement already satisfied: typer-slim in /usr/local/lib/python3.12/dist-packages (from transformers) (0.23.1)
Requirement already satisfied: safetensors>=0.4.3 in /usr/local/lib/python3.12/dist-packages (from transformers) (0.7.0)
Requirement already satisfied: tqdm>=4.27 in /usr/local/lib/python3.12/dist-packages (from transformers) (4.67.3)
Requirement already satisfied: JPype1>=0.7.0 in /usr/local/lib/python3.12/dist-packages (from konlpy) (1.7.0)
Requirement already satisfied: lxml>=4.1.0 in /usr/local/lib/python3.12/dist-packages (from konlpy) (6.0.3)
Requirement already satisfied: requests<3,>=2.27 in /usr/local/lib/python3.12/dist-packages (from gtts) (2.32.4)
Requirement already satisfied: click<8.2,>=7.1 in /usr/local/lib/python3.12/dist-packages (from gtts) (8.1.8)
Requirement already satisfied: fsspec>=2023.5.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>=1.3.0->transformers) (2025.3.0)
Requirement already satisfied: hf-xet<2.0.0,>=1.4.3 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>=1.3.0->transformers) (1.4.3)
Requirement already satisfied: httpx<1,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>=1.3.0->transformers) (0.28.1)
Requirement already satisfied: typer in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>=1.3.0->transformers) (0.23.1)
Requirement already satisfied: typing-extensions>=4.1.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>=1.3.0->transformers) (4.15.0)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.27->gtts) (3.4.7)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.27->gtts) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.27->gtts) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.27->gtts) (2026.2.25)
Requirement already satisfied: anyio in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-hub<2.0,>=1.3.0->transformers) (4.13.0)
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-hub<2.0,>=1.3.0->transformers) (1.0.9)
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from httpcore==1.*->httpx<1,>=0.23.0->huggingface-hub<2.0,>=1.3.0->transformers) (0.16.0)
Requirement already satisfied: shellingham>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from typer->huggingface-hub<2.0,>=1.3.0->transformers) (1.5.4)
Requirement already satisfied: rich>=10.11.0 in /usr/local/lib/python3.12/dist-packages (from typer->huggingface-hub<2.0,>=1.3.0->transformers) (13.9.4)
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.12/dist-packages (from typer->huggingface-hub<2.0,>=1.3.0->transformers) (0.0.4)
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from rich>=10.11.0->typer->huggingface-hub<2.0,>=1.3.0->transformers) (4.0.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from rich>=10.11.0->typer->huggingface-hub<2.0,>=1.3.0->transformers) (2.20.0)
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-packages (from markdown-it-py>=2.2.0->rich>=10.11.0->typer->huggingface-hub<2.0,>=1.3.0->transformers) (0.1.2)
```

Import libraries

```
from transformers import MarianMTModel, MarianTokenizer
from konlpy.tag import Okt
from hangul_romanize import Transliter
from hangul_romanize.rule import academic
from gtts import gTTS
from IPython.display import Audio, display
```

Core Translation (Korean → English)

We'll use the pretrained model: Helsinki-NLP opus-mt-ko-en

```
model_name = "Helsinki-NLP/opus-mt-ko-en"

tokenizer = MarianTokenizer.from_pretrained(model_name)
model = MarianMTModel.from_pretrained(model_name)

def translate(text):
    inputs = tokenizer(text, return_tensors="pt", padding=True, truncation=True)
    translated = model.generate(**inputs)
    return tokenizer.decode(translated[0], skip_special_tokens=True)
```

```
/usr/local/lib/python3.12/dist-packages/transformers/models/marian/tokenization_marian.py:176: UserWarning: Recommended: pip install sacremoses.
  warnings.warn("Recommended: pip install sacremoses.")
Loading weights: 100%|| 258/258 [00:00<00:00, 546.90it/s, Materializing param=model.shared.weight]
The tied weights mapping and config for this model specifies to tie model.shared.weight to model.decoder.embed_tokens.weight, but both are presen
The tied weights mapping and config for this model specifies to tie model.shared.weight to model.encoder.embed_tokens.weight, but both are presen
```

Extract Important Korean Words

```
from konlpy.tag import Okt

okt = Okt()

def extract_words(text):
    pos = okt.pos(text)
    words = [word for word, tag in pos if tag in ['Noun', 'Verb', 'Adjective']]
    return list(set(words))
```

Romanization (Pronunciation)

```
transliter = Transliter(academic)

def romanize(word):
    return transliter.translit(word)
```

Voice Pronunciation

```
def speak(word):
    tts = gTTS(word, lang='ko')
    tts.save("audio.mp3")
    return Audio("audio.mp3", autoplay=False)
```

Input Korean Song Lyrics

```
korean_lyrics = """
여기까지가 끝인가 보오
이제 나는 돌아서겠소
억지 노력으로 인연을 거슬러 괴롭히지는 않겠소
"""
```

Translate Full Lyrics

```
english_translation = translate(korean_lyrics)

print("🔷 English Translation:\n")
print(english_translation)
```

```
🔷 English Translation:

Now I'm going to turn my back on you, and I'm not going to torture you back with my hard work.
```

Flashcard Generator Logic

For each extracted word:

Korean word, Romanization, English meaning (translate word), Example sentence (line from lyrics), Audio

Simple Streamlit App

```
words = extract_words(korean_lyrics)

print("\n🔷 Flashcards:\n")

for w in words[:10]:  # limit to 10 for demo
    print("================================")
    print("Korean Word:", w)
    print("Pronunciation:", romanize(w))
    print("Meaning:", translate(w))

    display(speak(w))
```

```
🔷 Flashcards:

================================
Korean Word: 억지
Pronunciation: eogji
Meaning: I don't think so.
```

0:01 / 0:01

```
================================
Korean Word: 이제
Pronunciation: ije
Meaning: Now.
```

0:00 / 0:00

```
================================
Korean Word: 나
Pronunciation: na
Meaning: Me.
```

0:00 / 0:00

```
================================
Korean Word: 보오
Pronunciation: bo-o
Meaning: Boo.
```

0:00 / 0:00

```
================================
Korean Word: 끝
Pronunciation: kkeut
Meaning: End
```

0:00 / 0:00

```
================================
Korean Word: 괴롭히지는
Pronunciation: goelobhijineun
```

Performance Evaluation

```
!pip install sacrebleu
```

```
Collecting sacrebleu
  Downloading sacrebleu-2.6.0-py3-none-any.whl.metadata (39 kB)
Collecting portalocker (from sacrebleu)
  Downloading portalocker-3.2.0-py3-none-any.whl.metadata (8.7 kB)
Requirement already satisfied: regex in /usr/local/lib/python3.12/dist-packages (from sacrebleu) (2025.11.3)
Requirement already satisfied: tabulate>=0.8.9 in /usr/local/lib/python3.12/dist-packages (from sacrebleu) (0.9.0)
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist-packages (from sacrebleu) (2.0.2)
Collecting colorama (from sacrebleu)
  Downloading colorama-0.4.6-py2.py3-none-any.whl.metadata (17 kB)
Requirement already satisfied: lxml in /usr/local/lib/python3.12/dist-packages (from sacrebleu) (6.0.3)
Downloading sacrebleu-2.6.0-py3-none-any.whl (100 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 100.8/100.8 kB 7.0 MB/s eta 0:00:00
Downloading colorama-0.4.6-py2.py3-none-any.whl (25 kB)
Downloading portalocker-3.2.0-py3-none-any.whl (22 kB)
Installing collected packages: portalocker, colorama, sacrebleu
Successfully installed colorama-0.4.6 portalocker-3.2.0 sacrebleu-2.6.0
```

```
test_data = [
    ("안녕하세요", "Hello"),
    ("나는 학생입니다", "I am a student"),
    ("오늘 날씨가 좋다", "The weather is nice today"),
    ("음악을 듣는 것을 좋아해요", "I like listening to music"),
    ("이 노래는 정말 아름답다", "This song is really beautiful"),
]
```

```
import sacrebleu

predictions = []
references = []

for ko, ref in test_data:
    pred = translate(ko)
    predictions.append(pred)
    references.append(ref)

bleu = sacrebleu.corpus_bleu(predictions, [references])
print("BLEU Score:", bleu.score)
```

```
BLEU Score: 18.35667934257646
```

Vocabulary Extraction Evaluation (Precision)

```
sentence = "나는 음악을 정말 좋아해요"
expected_words = {"나", "음악", "좋아하다"}

extracted = set(extract_words(sentence))

precision = len(extracted & expected_words) / len(extracted)
print("Precision:", precision)
```

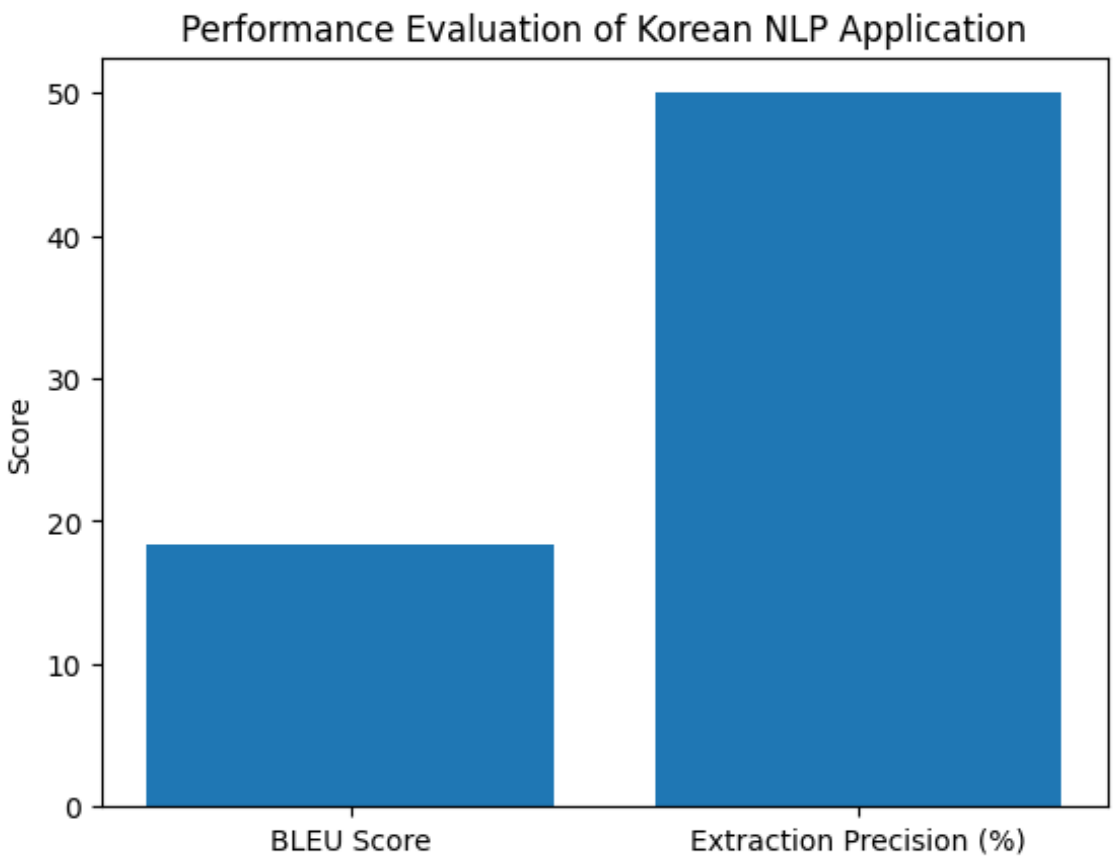
```
Precision: 0.5
```

Visualization

```
import matplotlib.pyplot as plt

labels = ['BLEU Score', 'Extraction Precision (%)']
values = [18.35, 50]

plt.bar(labels, values)
plt.title("Performance Evaluation of Korean NLP Application")
plt.ylabel("Score")
plt.show()
```


Performance Evaluation of Korean NLP Application
Score
0
10
20
30
40
50
BLEU Score
Extraction Precision (%)

Demo

```
!pip install -q streamlit pyngrok transformers sentencepiece konlpy hangul-romanize gtts
```

```
from pyngrok import ngrok
ngrok.set_auth_token("3ByfW0Be8GQeb9raeQuIwtJUMD1_6me6LYTuEkvubfgZhjRiq")
```

```
%%writefile app.py
import streamlit as st
from transformers import MarianMTModel, MarianTokenizer
from konlpy.tag import Okt
from hangul_romanize import Transliter
from hangul_romanize.rule import academic
from gtts import gTTS
import os

st.set_page_config(page_title="Korean Lyrics Flashcards", layout="centered")
st.title("🎵 Korean Lyrics Translator & Flashcards")

@st.cache_resource
def load_models():
    model_name = "Helsinki-NLP/opus-mt-ko-en"
    tokenizer = MarianTokenizer.from_pretrained(model_name)
    model = MarianMTModel.from_pretrained(model_name)
    return tokenizer, model

tokenizer, model = load_models()

def translate(text):
    inputs = tokenizer(text, return_tensors="pt", padding=True, truncation=True)
    translated = model.generate(**inputs)
    return tokenizer.decode(translated[0], skip_special_tokens=True)

okt = Okt()
transliter = Transliter(academic)

def extract_words(text):
    pos = okt.pos(text)
    return list(dict.fromkeys([w for w,t in pos if t in ['Noun','Verb','Adjective']]))

def romanize(word):
    return transliter.translit(word)

def generate_audio(word):
    filename = f"{word}.mp3"
    if not os.path.exists(filename):
        gTTS(word, lang='ko').save(filename)
    return filename

if "cards" not in st.session_state:
    st.session_state.cards = []
    st.session_state.index = 0
    st.session_state.translation = ""

lyrics = st.text_area("Paste Korean Lyrics Here")

if st.button("Generate Flashcards"):
    st.session_state.translation = translate(lyrics)

    words = extract_words(lyrics)[:10]
    cards = []

    for w in words:
        cards.append({
            "word": w,
            "pron": romanize(w),
            "meaning": translate(w),
            "audio": generate_audio(w)
        })

    st.session_state.cards = cards
    st.session_state.index = 0
```

```
if st.session_state.translation:
    st.subheader("📘 English Translation")
    st.write(st.session_state.translation)

if st.session_state.cards:
    card = st.session_state.cards[st.session_state.index]

    st.markdown("---")
    st.markdown(f"## 🟦 {card['word']}")
    st.write("**Pronunciation:**", card['pron'])
    st.write("**Meaning:**", card['meaning'])
    st.audio(card['audio'])

    col1, col2 = st.columns(2)

    with col1:
        if st.button("⬅ Previous") and st.session_state.index > 0:
            st.session_state.index -= 1

    with col2:
        if st.button("Next ➡") and st.session_state.index < len(st.session_state.cards)-1:
            st.session_state.index += 1
```

```
Overwriting app.py
```

```
!pkill ngrok
!pkill streamlit
```

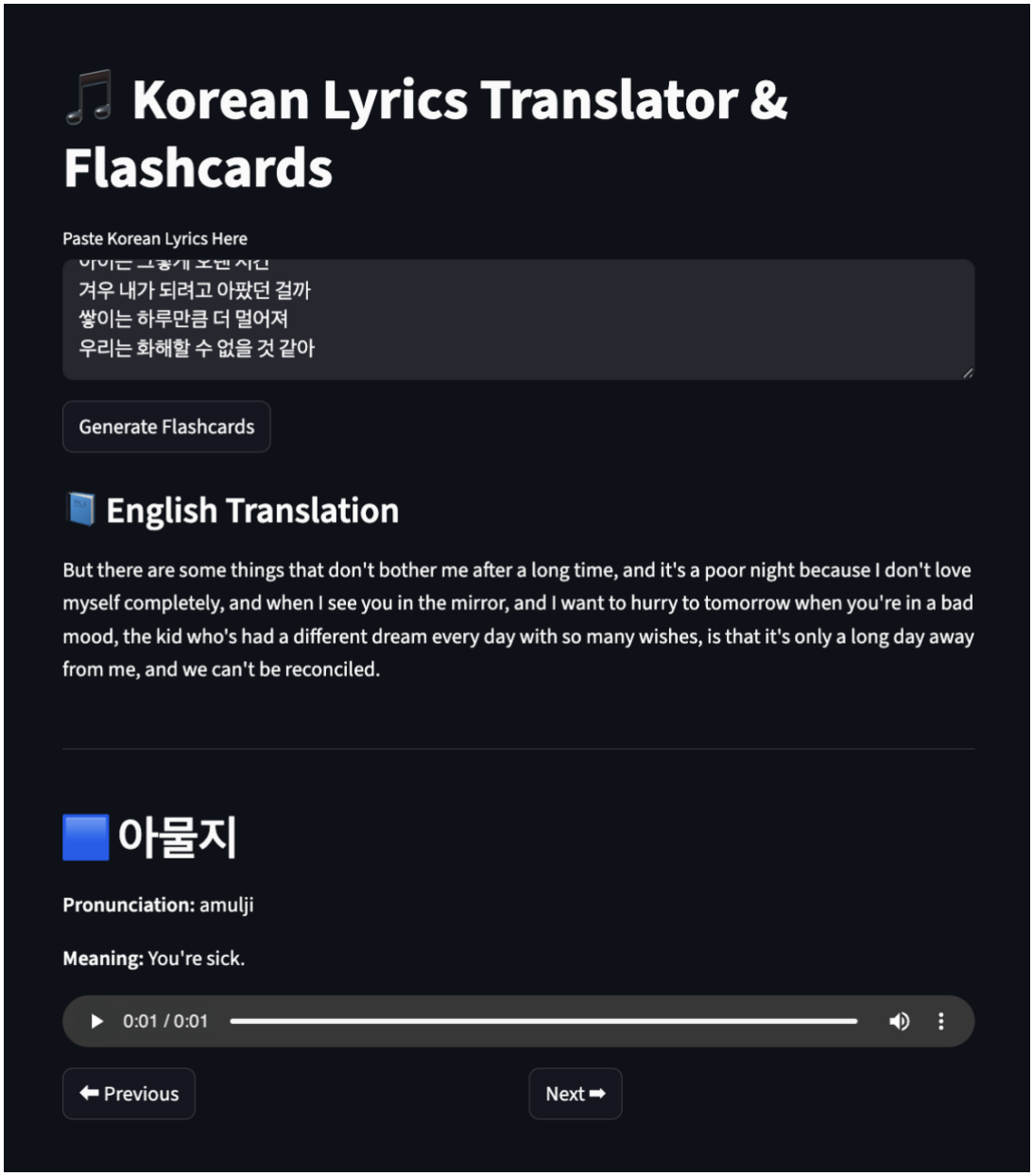
```
WARNING:pyngrok.process.ngrok:t=2026-04-14T23:21:10+0000 lvl=warn msg="Stopping forwarder" name=
WARNING:pyngrok.process.ngrok:t=2026-04-14T23:21:10+0000 lvl=warn msg="Error restarting forwarde
```

```
import time
get_ipython().system_raw('streamlit run app.py &')
time.sleep(6)
```

```
from pyngrok import ngrok

ngrok.kill()  # important
public_url = ngrok.connect(8501)
print(public_url)
```

```
NgrokTunnel: "https://opal-flooded-unpreposterously.ngrok-free.dev" -> "http://localhost:8501"
```



Conclusion

The mini-project successfully demonstrates how multiple NLP components machine translation, morphological analysis, vocabulary extraction, romanization, and speech synthesis can be integrated into an interactive educational application.

Despite moderate BLEU scores due to the poetic nature of song lyrics, real-time validation with an IU song showed translations closely matching Google Translate, confirming semantic reliability. The vocabulary flashcards with pronunciation and audio further enhance the learning experience.

This project proves that NLP techniques can be effectively applied to real-world language learning scenarios, resulting in a practical, user-friendly system that combines translation accuracy with interactive vocabulary learning.